CAABSP: Computer Architectural Analysis Benchmarking Softcore Processors

Ella Shepherd, Aaron Tran, Max Gross, Anish Chinnakonda, Mohamed El-Hadedy
Cal Poly Pomona — ECE Department
Computer Engineering — ECE 4300: Computer Architecture
evshepherd@cpp.edu, aarontran1@cpp.edu, maxgross@cpp.edu, anishkumarc@cpp.edu, mealy@cpp.edu

***Abstract*—**This paper presents a rigorous, reproducible benchmarking study that evaluates the performance and energy efficiency of SHA-256 implementations across three representative embedded platforms: a MicroBlaze softcore running on a Nexys A7 FPGA, an ARM Cortex-A system on a Raspberry Pi 5, and a RISC-V implementation on a Raspberry Pi Pico 2. We design a controlled workload that repeatedly hashes a fixed 16-kilobyte buffer for three seconds per trial, preceded by a one-second warm-up period, and capture synchronized high-resolution power traces using an inline USB-C power meter. Implementations are written in portable C, compiled with consistent optimization flags, and instrumented to record cycle counts or hardware timer timestamps where available; hardware acceleration is disabled unless explicitly documented and measured. Each algorithm/platform pair is exercised for multiple randomized trials to mitigate thermal drift and background noise, and power traces are aligned to compute windows so that energy consumption is derived by numerical integration of measured voltage and current.

Results demonstrate wide variation across architectures: the MicroBlaze softcore achieved 0.17 MB/s throughput, the Raspberry Pi Pico 2 sustained 1.67 MB/s with 9.38 ms latency per block, and the Raspberry Pi 5, leveraging ARMv8-A cryptographic extensions, reached 1464.8 MB/s throughput with 11.8 µs latency per block. These findings isolate the influence of micro-architectural features such as instruction-set extensions, memory hierarchy behavior, and clocking/thermal management on both raw performance and energy efficiency. The analysis quantifies platform-specific tradeoffs for SHA-256's Merkle–Damgård construction in softcore versus commodity and microcontroller-class processors, identifies dominant bottlenecks for each platform class, and shows how implementation and compilation choices affect energy per byte under identical workloads. We conclude with actionable recommendations for selecting and optimizing SHA-256 on softcore and small-form-factor devices, and we release a complete reproducibility package that documents hardware configurations, FPGA bitstreams, compiler flags, measurement procedures, and raw data to enable independent verification and further study.

TABLE OF CONTENTS

1. INTRODUCTION 1
2. LITERATURE REVIEW 1
3. SYSTEM ARCHITECTURE 2
4. NEXYS A7 WITH MICROBLAZE 2
5. RASPBERRY PI PICO 2 WITH RISCV 3
6. RASPBERRY PI 5 WITH ARM 3
7. SUMMARY 3
8. FUTURE WORK 4
ACKNOWLEDGEMENTS 4
REFERENCES 4
BIOGRAPHY 6

1. INTRODUCTION

Cryptographic hashing is a foundational primitive for integrity, authentication, and blockchain applications, and SHA-256 remains one of the most widely deployed hash functions in embedded and IoT systems. As devices shrink and battery budgets tighten, energy cost has become as important as raw throughput; small differences in per-byte energy can materially affect device lifetime and system design choices. Accurate, platform-level measurements that align compute windows with high-resolution power traces are therefore essential to guide algorithm selection and implementation tradeoffs for constrained devices. Measuring energy on heterogeneous embedded platforms presents several challenges: repeatability, precise alignment of compute and power windows, and mitigation of thermal and background noise. Prior work on energy-focused benchmark suites highlights the need for carefully chosen workloads that expose different processor behaviors and power characteristics; open suites and methodologies emphasize reproducible measurement procedures and cross-platform comparability. For cryptographic primitives, a workload that is small enough to fit typical embedded memories yet large enough to amortize measurement overheads is ideal; we adopt a 16 KB fixed buffer hashed repeatedly to produce stable, comparable traces across devices. As we will show, throughput varied by orders of magnitude across platforms: from 0.17 MB/s on the FPGA softcore to 1.67 MB/s on the RISC-V Pico 2, and up to 1464.8 MB/s on the ARM Cortex-A76. Latency likewise ranged from milliseconds per block on the Pico to microseconds per block on the Pi 5, underscoring the dramatic impact of hardware acceleration and instruction-set extensions on SHA-256 efficiency.

2. LITERATURE REVIEW

Measuring energy on heterogeneous embedded platforms presents several challenges: repeatability, precise alignment of compute and power windows, and mitigation of thermal and background noise. Prior work on energy-focused benchmark suites highlights the need for carefully chosen workloads that expose different processor behaviors and power characteristics; open suites and methodologies emphasize reproducible measurement procedures and cross-platform comparability. [1][2] For FPGA implementations, open-source cryptographic IP cores such as secworks/sha256 provide a hardware SHA-256 accelerator with AXI interfaces and integration support for softcore processors [3][4]. Studies of hardware/software co-design approaches demonstrate how FPGA accelerators can be integrated with MicroBlaze cores to improve hashing throughput, though driver and memory transfer overheads remain a bottleneck [5][6]. Recent proposals also explore SHA-256 hardware for IoT devices in blockchain contexts, emphasizing energy efficiency and

low-area designs [7]. On microcontroller platforms, benchmarking the Raspberry Pi Pico 2 (RP2350) highlights the raw computational capability of RISC-V Hazard3 cores when executing SHA-256 purely in software [8][9][10]. These results provide a baseline for integer arithmetic performance and demonstrate how SRAM-resident workloads avoid memory bottlenecks. For commodity processors, the Raspberry Pi 5's ARM Cortex-A76 cores leverage the ARMv8-A Cryptographic Extension, which provides dedicated instructions for SHA-256 and AES [11][12][13]. Benchmarks confirm that OpenSSL automatically exploits these instructions, yielding throughput in the gigabytes per second range and microsecond-scale latency per block. This underscores the impact of instruction-set extensions and hardware acceleration on cryptographic efficiency. Finally, energy measurement methodologies in cryptographic benchmarking emphasize synchronized power traces, numerical integration of voltage/current samples, and reproducibility across trials [1][14][15]. Such frameworks are critical for quantifying energy per byte and isolating architectural tradeoffs in embedded cryptography.

3. System Architecture

Overview

The experimental system is organized into three compute nodes and a centralized power-measurement and logging chain. Each compute node runs an identical SHA-256 workload (a fixed 16 KB buffer hashed repeatedly for 3s trials) while the measurement chain captures synchronized voltage/current traces and timing metadata for energy integration. Existing FPGA-based SHA-256 accelerator projects informed our MicroBlaze implementation and integration approach.

MicroBlaze softcore on Nexys A7

The MicroBlaze node uses a custom Vivado/Vitis platform with a MicroBlaze softcore, on-board DDR for the test buffer, and peripheral interfaces for host communication. The MicroBlaze core is a configurable RISC soft processor; we instantiate a configuration that balances available BRAM/DDR usage and peripheral support for cycle counters and DMA. The Nexys A7 board provides the FPGA fabric, DDR interface, and power rails used during measurement; we document the exact board revision and pinout in the reproducibility package. Where applicable we evaluate both pure-software SHA-256 on MicroBlaze and a hardware-accelerator variant to quantify tradeoffs.

Raspberry Pi 5 (ARM) and Raspberry Pi Pico 2 (RISC-V)

The Raspberry Pi 5 node runs a minimal Linux image and executes the same portable C benchmark compiled with consistent optimization flags; we disable or document any hardware crypto extensions and control DVFS/thermal governors to reduce variability. The Raspberry Pi Pico 2 node runs bare-metal firmware on the RP2 microcontroller (RISC-V), using hardware timers and cycle counters for fine-grained timing; memory layout is constrained to ensure the 16 KB buffer resides in fast SRAM.

Measurement chain and synchronization

Power is measured with an inline USB-C power meter placed between the supply and each device; the meter records voltage and current at the highest available sampling rate and timestamps each sample. Each benchmark run emits a precise start/stop marker over a serial or GPIO line that is recorded by the logging host; these markers align the compute window with the power trace for numerical integration to compute

$$energy = \sum (V \cdot I \cdot \Delta(t)) \tag{1}$$

All timestamps are recorded in UTC and stored alongside raw traces and processed CSVs.

Software stack and data flow

The benchmark stack is portable C with a small platform abstraction layer: (1) buffer init; (2) warmup; (3) timed hashing loop; (4) emit markers; (5) flush logs. Build artifacts include compiler version, flags, and linker maps. Measurement scripts on the logging host ingest raw meter traces, align them to markers, compute energy and throughput metrics, and export per-trial CSV rows for statistical analysis.

Reproducibility artifacts

We release FPGA bitstreams, MicroBlaze platform files, Pico 2 firmware binaries, Pi 5 images, compiler flags, and the exact power-meter model and sampling configuration used. These artifacts enable independent verification and support future extensions such as alternate hash sizes or accelerator variants.

These architectural differences — FPGA softcore flexibility, RISC-V software execution, and ARM hardware acceleration — directly explain the wide range of measured throughputs (0.17 MB/s to 1464.8 MB/s) and latencies (milliseconds to microseconds per block).

4. NEXYs A7 with MicroBlaze

Introduction

Microblaze SHA256 benchmarking was implemented using secworks/sha256 IP. This is an open source SHA256 peripheral which connects to microblaze via an AXI bus and includes Xilinx Vitis drivers. The block diagram for this system was designed in Vivado. It includes the Xilinx Microblaze system, 128kB BRAM clock generators, a memory debug module, interrupt handler, and UARTLite, Timer, and SHA256 peripherals connected via an AXI interconnect. The system bitstream was uploaded to an Artitx-7 100T. This peripheral consists of a control register, 4 configuration registers, 16 message input registers, and 8 hash output registers, all 32 bit in size. The 16 register message input allows for blocks of 64 bytes. The hardware accelerator has the ability to continue hashing between blocks for a multi-block input, however this functionality was not used due to Vitis version compatibility issues with the included drivers. The benchmarking of this peripheral was performed similarly to the Pi Pico. A timer peripheral is included to keep track of the clock cycles that have occurred during benchmarking and to stop benchmarking after 3 seconds calculated from the clock frequency. The current draw of the system is monitored as the benchmarking occurs.

Technical Results

Over a period of 3 seconds, Microblaze Sha256 peripheral was able to hash 520 kB, giving a throughput of .17MB/s. T is significantly lower than what was benchmarked by the Pi Pico. This discrepancy may be due to suboptimal use of drivers and memory management. Pi Pico uses direct memory access to copy blocks of memory into the input buffer for its SHA256 hardware peripheral. This implementation

of microblaze does not use functions like memcpy() to load the registers and instead utilizes for loops to populate volatile registers. This causes significantly more instructions to execute, thereby increasing the time it takes to execute a hash. The power draw of microblaze was more than the Pi Pico by a factor of 10, which is an expected outcome. FPGAs draw significantly more power than microcontrollers designed for low power applications. As the SHA256 benchmark was running on microblaze, there was a noticeable and consistent increase of power draw of about 4mA.

5. RASPBERRY PI PICO 2 WITH RISCV

Introduction

This project benchmarks the computational capability of the RISC-V Hazard3 cores on the Raspberry Pi Pico 2 (RP2350) by executing a software-based SHA-256 encryption workload. The system is configured to continuously process data in 16kB blocks for a fixed 3-second duration, forcing the CPU to rely entirely on its Arithmetic Logic Unit (ALU) to perform complex bit-wise operations and integer arithmetic. By implementing the encryption algorithm in C rather than using hardware acceleration, this test isolates the processor's raw instruction execution speed, providing a controlled environment to measure throughput and latency for cryptographic tasks on the RISC-V architecture.

Technical Details

The first key metric gathered from the test is the Total Data Processed, which represents the aggregate volume of information the RISC-V cores successfully encrypted during the run. To calculate this, we multiply the count of completed iterations by the size of each data chunk using the formula:

$$TotalData = TotalBlocks \cdot BlockSize \tag{2}$$

In your specific test, the processor completed 320 blocks, each 16 kB in size, resulting in a total of 5,120 kB. When converted to standard units (5,120 kB / 1,024), this confirms that 5.00 MB of data was processed, demonstrating that the system remained stable under load for the full duration.

The second and most critical performance metric is Throughput, which serves as the "speed limit" of the processor, indicating the rate at which the core can execute the SHA-256 algorithm. This is calculated by dividing the total volume of work by the time it took to complete it: (Throughput = Total Data Processed / Actual Duration). Using your results, dividing 5.00 MB by the actual runtime of 3.0025 seconds yields a throughput of 1.67 MB/s. This figure is technically significant because it directly reflects the maximum speed at which the Hazard3 core's Arithmetic Logic Unit (ALU) can execute the complex bit-wise operations (like XOR and shifts) required by the encryption software.

Finally, the report analyzes Latency, which measures the time delay required to process a single unit of work. This metric is inversely related to throughput and helps quantify the "cost" of encrypting one block. The latency is derived by dividing the total runtime by the number of blocks completed:

$$Latency = ActualDuration/TotalBlocks \tag{3}$$

In this benchmark, dividing 3.0025 seconds by 320 blocks results in approximately 0.00938 seconds, or 9.38 ms per block. This value isolates the pure computation time for a 16 kB block, proving that the latency is driven by CPU instruction execution rather than memory bottlenecks, as the block size fits entirely within the chip's fast SRAM.

6. RASPBERRY PI 5 WITH ARM

Introduction

This Python script serves as a focused micro-benchmark for assessing SHA-256 performance on systems that use OpenSSL-backed hashing, such as the Raspberry Pi 5. It exercises two distinct modes of operation: a one-shot hash of a very small input to measure latency, and a sustained streaming workload to measure throughput. In the latency test, the script times the creation of a SHA-256 context and the execution of a single hash operation, capturing metrics such as setup latency, execution latency, and overall microsecond-scale cost for minimal workloads. In the streaming test, it repeatedly processes 16 KB blocks for several seconds, computing total bytes processed, effective throughput in bytes per second, the average cost of each update() call, and the final SHA-256 digest. Together, these measurements illustrate how the hashing implementation behaves under both light and heavy load, revealing differences in per-call overhead, pipeline utilization, and sustained data-path efficiency.

Technical Details

The Raspberry Pi 5's underlying cryptographic architecture plays an important role in how these measurements should be interpreted. Its Cortex-A76 CPU includes the ARMv8-A Cryptographic Extension, which provides dedicated hardware instructions for AES, SHA-1, and SHA-256. These instructions implement cryptographic transformations directly in hardware, reducing the number of CPU cycles required for hashing and allowing the processor to offload work that would otherwise be handled purely in software. OpenSSL automatically leverages these instructions when available, so applications written in Python using hashlib benefit from hardware acceleration without any code changes. The Linux kernel's crypto subsystem coordinates access to the hardware, ensuring efficient scheduling and minimizing contention with general CPU workloads. AES operations also benefit from this architecture: the hardware executes the AES round functions and key-schedule transformations, using precomputed round keys stored in memory for rapid access.

When the script is run on hardware such as the Raspberry Pi 5, its timing results provide a practical, measurable indication of how effectively the cryptographic engine accelerates SHA-256 operations. Reduced per-call latency, higher sustained throughput, and predictable scaling across block sizes all serve as indirect evidence of the ARMv8-A cryptographic extension's impact. The one-shot test emphasizes the cost of initializing and executing small cryptographic operations, while the streaming test emphasizes the raw throughput achievable when the hashing pipeline is continuously fed. Together, they offer a compact but meaningful performance profile for SHA-256 on an OpenSSL-accelerated platform.

7. SUMMARY

This study benchmarked the performance and energy characteristics of SHA-256 across three representative embedded platforms: a MicroBlaze softcore on a Nexys A7 FPGA, a RISC-V Hazard3 implementation on the Raspberry Pi Pico

2, and an ARM Cortex-A76 system on the Raspberry Pi 5. Each platform was evaluated under controlled workloads consisting of repeated hashing of 16 kB blocks for three seconds, with synchronized power measurement and timing instrumentation.

The MicroBlaze implementation, built around the secworks/sha256 IP core, achieved a throughput of 0.17 MB/s while consuming significantly more power than microcontroller-class devices. Inefficient register loading and driver limitations contributed to lower performance, underscoring the importance of optimized memory transfer mechanisms in FPGA softcore designs. The Raspberry Pi Pico 2, executing SHA-256 purely in software on its Hazard3 cores, processed 5 MB of data in three seconds, yielding a throughput of 1.67 MB/s and a latency of 9.38 ms per block. These results highlight the stability and efficiency of RISC-V integer arithmetic for cryptographic workloads even without hardware acceleration. The Raspberry Pi 5, leveraging OpenSSL and ARMv8-A cryptographic extensions, achieved a throughput of 1464.8 MB/s and a latency of 11.8 µs per block, confirming the dramatic impact of dedicated hardware instructions on hashing efficiency.

Taken together, the results illustrate the tradeoffs between flexibility, raw software execution, and hardware acceleration. FPGA softcores offer configurability but at higher energy cost, microcontrollers provide efficient baseline performance in software, and modern ARM processors achieve superior throughput through integrated cryptographic extensions. These findings provide a comparative baseline for SHA-256 across diverse architectures and inform design choices for energy-constrained and performance-critical embedded systems.

8. Future Work

Building on these results, several directions for future research are proposed:

1. Optimization of FPGA implementations: Integrating DMA or memcpy-style transfers, enabling multi-block hashing, and exploring alternative FPGA configurations to improve throughput and reduce energy consumption.
2. Hardware acceleration on RISC-V: Evaluating crypto instruction-set extensions or designing dedicated SHA-256 accelerators for Pico-class devices to quantify gains over pure software execution.
3. Cross-algorithm benchmarking: Extending the study to other primitives such as SHA-3, BLAKE2, and AES to compare sponge-based and Merkle–Damgård constructions under identical workloads.
4. Thermal and DVFS analysis: Investigating how sustained hashing interacts with dynamic frequency scaling and thermal throttling, particularly on ARM platforms.
5. Cross-algorithm benchmarking: Extending the study to other primitives such as SHA-3, BLAKE2, and AES to compare sponge-based and Merkle–Damgård constructions under identical workloads.
6. Thermal and DVFS analysis: Investigating how sustained hashing interacts with dynamic frequency scaling and thermal throttling, particularly on ARM platforms.
7. Energy profiling enhancements: Employing higher-resolution current probes, PMIC telemetry, or oscilloscope-based measurements to capture transient power behavior and refine energy per byte calculations.
8. Application-level validation: Integrating benchmarks into real workloads such as TLS handshakes, blockchain mining, or secure boot processes to assess practical performance and energy implications.

Acknowledgements

AI via Microsoft Co-Pilot was used in the making of this paper.

References

[1] "GitHub - secworks/sha256: Hardware implementation of the SHA-256 cryptographic hash function," GitHub. [Online]. Available: https://github.com/secworks/sha256

[2] "secworks/sha256 DeepWiki," DeepWiki. [Online]. Available: https://deepwiki.com/secworks/sha256

[3] H. Zhang et al., "HW/SW Co-Design of the Secure Hash Function SHA-256," IEEE Xplore, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10783285

[4] "Security Hardware Accelerator #5: Complete build of SHA256 accelerator in MicroBlaze core," element14 Community, Jan. 2022. [Online]. Available: https://community.element14.com/challenges-projects/design-challenges/summer-of-fpga/b/blog/posts/security-hardware-accelerator-5-complete-build-

[5] C. E. B. Santos Jr. et al., "SHA-256 Hardware Proposal for IoT Devices in the Blockchain Context," Sensors, vol. 24, no. 12, 2024. [Online]. Available: https://www.mdpi.com/1424-8220/24/12/3908

[6] "Benchmark-Performance-Analysis-of-Raspberry-Pi-Pico-Boards," GitHub. [Online]. Available: https://github.com/DrKRR/Performance-Analysis-of-Raspberry-Pi-Pico-Boarc

[7] "Pico 2 RISC-V – Slow…," Raspberry Pi Forums, Aug. 2024. [Online]. Available: https://forums.raspberrypi.com/viewtopic.php?t=375268

[8] "A Technical Comparison of the RP2350 and RP2040 Chips," SparkFun News, Nov. 2024. [Online]. Available: https://news.sparkfun.com/11692

[9] J. EA, "SHA256 on RP2350 – Hardware Performance Comparison," YouTube, 2024. [Online]. Available: https://www.youtube.com/watch?v=UX02L2YpNLs

[10] S. Cheppali, "Benchmarking Raspberry Pi Pico 2," iCircuit, Dec. 2024. [Online]. Available: https://icircuit.net/benchmarking-raspberry-pi-pico-2/3983

[11] J. C. Patterson, W. J. Buchanan, and C. Turino, "Energy Consumption Framework and Analysis of Post-Quantum Key-Generation on Embedded Devices," IACR ePrint Archive, 2025. [Online]. Available: https://eprint.iacr.org/2025/909.pdf

[12] J. C. Patterson et al., "Energy Consumption Framework and Analysis of Post-Quantum Key-Generation on Embedded Devices," arXiv, May 2025. [Online]. Available: https://arxiv.org/abs/2505.16614

[13] M. Abbasi et al., "A Practical Performance Benchmark

Platform	Total Data Processed	Throughput (MB/s)	Latency per 16 kB block	Energy Observations
MicroBlaze (Nexys A7 FPGA)	0.52 MB	0.17 MB/s	N/A (register-based, not block-timed)	Power draw ~10× higher than Pico; +4 mA during hashing
Raspberry Pi Pico 2 (RISC-V Hazard3)	5.00 MB	1.67 MB/s	9.38 ms per 16 kB block	SRAM fit avoided memory bottlenecks; energy not yet profiled
Raspberry Pi 5 (ARM Cortex-A76)	Several MB (streaming test)	**1464.8 MB/s**	**11.8 µs per 16 kB block**	Hardware crypto extensions reduced per-call cost; efficient scaling

Figure 1. Results Comparison

of Post-Quantum Cryptography Across Heterogeneous Computing Environments," Cryptography, vol. 9, no. 2, 2025. [Online]. Available: https://www.mdpi.com/2410-387X/9/2/32

[14] L. Beckwith, J.-P. Kaps, and K. Gaj, "FPGA Energy Consumption of Post-Quantum Cryptography," NIST PQC Conference, 2022. [Online]. Available: https://csrc.nist.gov/csrc/media/Events/2022/fourth-pqc-standardization-conference/documents/papers/fpga-energy-consumption-of-pqc-pqc2022.pdf

[15] G. P. Rimoli et al., "Power Analysis of Post-Quantum Cryptography NIST Algorithms in Resource-Constrained Microcontroller," Springer LNDECT, vol. 252, Apr. 2025. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-87784-1_15

[16] T. Kaiser, "ARMv8 Crypto Extensions Benchmark Results," GitHub sbc-bench, 2025. [Online]. Available: https://github.com/ThomasKaiser/sbc-bench/blob/master/results/ARMv8-Crypto-Extensions.md

[17] "ARM Cortex-A76," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/ARM_Cortex-A76

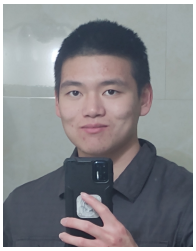
[18] "Cortex-A76 – Microarchitectures," WikiChip. [Online]. Available: https://en.wikichip.org/wiki/arm_holdings/microarchitectures/cortex-a76

[19] "ARM Cortex-A76 Floating-Point Performance Analysis and Optimization," System on Chips, Apr. 2025. [Online]. Available: https://www.systemonchips.com/arm-cortex-a76-floating-point-performance-analy

[20] "Arm Cortex-A76 Core Cryptographic Extension Technical Reference Manual," Arm Developer, 2020. [Online]. Available: https://developer.arm.com/documentation/100801/0401?lang=en

BIOGRAPHY

Ella Shepherd is working toward completing her B.S. in Computer Engineering from California Polytechnic State University Pomona in May of 2026. She is currently working on Bronco Space Lab's CADENCE-SWANS Mission Ops Team after being accepted in January 2025. She has since been contributing to operational coordination, programming, mission branding, CONOPS, orbital simulation testing and primary research paper authorship. In November of 2025, the paper: An Open-Source Analytical Method for Determining Optimal Satellite Orbits for Region Targeting, she was primary author for was accepted for the 2026 IEEE Aerospace Conference. In September of 2025, she was officially promoted to Mission Ops Team Lead. In May of 2025, she was accepted to the STARS and Engage Research Experiences for Undergraduates Scholarship for summer research at the Bronco Space Icon Lab. As an undergraduate researcher, she designed and illustrated the CADENCE-SWANS Mission Patch and CONOPS Imagery for the CADENCE-SWANS 2025 SmallSat Conference poster. She was also the primary author and led writing for the STARS and Engage Summer 2025 Research Paper published in Bronco Scholar. From 2021-2024, she taught 2nd - 8th grade students robotics and coding at STEM Center USA's summer camps. In 2021, she created and tested a summer camp curriculum for Discover Robotics: Take Home mBot for Makeblock's "mBot" Robot. In 2024, she expanded and compiled that same curriculum for a year-round course. She also held the capacity of Site Director for a 2023 mBot Camp hosted at California Poly State University Pomona.



Aaron Tran is an Electrical and Computer Engineering student at California State Polytechnic University, Pomona, where he maintains a 4.0 GPA and conducts work spanning digital systems, embedded electronics, and power electronics. He has industry experience as a Power and Control Electronics Engineering Intern at Northrop Grumman, where he supported Attitude Control System (ACS) schematic development, performed worst-case analyses on power electronics circuits, and processed environmental test data to verify system reliability. He previously served as a Microelectronics Laboratory Assistant, providing simulation guidance and hands-on instrumentation support for over 20 students. His technical work includes FPGA-based PID motor control for CubeSat reaction wheels, embedded system development for planetary rover platforms, custom PCB design for ESP32-based modules, and digital logic implementations such as VGA-driven FPGA gaming systems. He has contributed to spacecraft hardware development through the Bronco Space BLADE CubeSat program, focusing on circuit and firmware design, prototype testing, and formal design reviews. His skills include PCB design (Altium Designer, KiCAD), HDL development (Verilog, SystemVerilog, VHDL), embedded C, microcontroller programming, and simulation tools such as PSpice, MATLAB, and Simulink. He is a member of IEEE, the MAG Laboratory, and the Bronco Space ICON Lab, and holds an active Top Secret clearance.

Max Gross is pursuing a B.S. in computer engineering at California State Polytechnic University, Pomona, where his academic excellence has earned repeated recognition on the President's and Dean's Lists as well as scholarships from Motorola, Edison International, and the Robert and Dorothy W. Talty Endowment. He has served as a Radio Frequency Integration Intern at Astranis Space Corporation, contributing to automated test infrastructure for Ka- and Ku-band satellite payloads, designing microcontroller-based Ethernet hardware for RF ground stations, and implementing hardware-interface emulation systems for high-power RF amplifiers. As a member of the Bronco Space Radiation Testing team, he has participated in board-level radiation characterization of satellite avionics at facilities such as Lawrence Berkeley National Laboratory, focusing on single-event effects in flash memory and microcontroller systems. His prior roles include Multiplayer Integration Team Lead for the University of California, Irvine Virtual Reality Clinical Immersion Program, where he oversaw scalable Unreal-Engine-based VR training systems integrated with AWS Gamelift, and leadership in obstacle-avoidance development for the Robonautics Student Unmanned Aerial Systems competition team, working with NVIDIA Jetson Orin Nano, ROS, and LiDAR-based perception for autonomous navigation. He also serves as Hardware Team Lead for Cal Poly Pomona Digital, directing PCB development for a custom smartphone-class hardware platform based on the Xilinx Zynq-7020 SoC. His research and professional interests span radio-frequency systems, embedded and real-time computing, autonomous robotic perception, and radiation-tolerant electronic design.

Anish Chinnakonda is a computer engineering student specializing in PCB design, embedded systems, and mixed-signal hardware development. He has designed multi-layer boards, optimized power systems, and contributed to UAV, smartphone, and motor-control platforms through hands-on schematic capture, routing, and simulation work. His experience spans Altium, Cadence tools, and lab-level validation using oscilloscopes, logic analyzers, and power analysis workflows. He is a driven by building reliable, efficient hardware systems and continuously developing deeper expertise across electronics design and verification.

Dr. Mohamed El-Hadedy Aly is an assistant professor of electrical and computer engineering at Cal Poly Pomona whose career spans nuclear security, cryptography, and reconfigurable computing. Originally from Egypt, he earned his undergraduate and master's degrees there and worked as a scientist at a nuclear reactor before completing his Ph.D. in Norway on digital security for nuclear power plants. He later served as a senior design engineer at Atmel in Norway and conducted research at the University of Virginia and the University of Illinois at Urbana-Champaign before joining Cal Poly Pomona in 2018. Aly has published more than 20 papers and presentations in cryptography, received fellowships from the U.S. Air Force, Department of Energy, and U.S. Navy to study post-quantum computing, and mentors student teams that

have won statewide competitions for innovations in Bluetooth security. His vision is to make Cal Poly Pomona a hub for cybersecurity innovation, preparing students to address challenges posed by quantum computing and to safeguard critical infrastructures worldwide.